memfile.dat

00500113
00c00193
ff718393
0023e233
0041f2b3
004282b3
02728663
0041a233
00020463
00000293
0023a233
00001397 //changed
402383b3
0471a223
05002103
008001ef
00100113
00310133
04202a23

top.sv

```
module flopr #(parameter WIDTH = 8)
(input logic clk, reset,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= d;
endmodule
module flopr2 #(parameter WIDTH = 8)
(input logic clk, reset, enable,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else if (enable) q <= d;
endmodule
module mux2 #(parameter WIDTH = 8)
(input logic [WIDTH-1:0] d0, d1,
 input logic s,
 output logic [WIDTH-1:0] y);
    assign y = s ? d1 : d0;
endmodule
module mux3 #(parameter WIDTH = 8)
(input logic [WIDTH-1:0] d0, d1, d2,
 input logic [1:0] s,
 output logic [WIDTH-1:0] y);
    assign y = s[1] ? d2 : (s[0] ? d1 : d0);
endmodule
module alu(input logic [31:0] a, b,
           input logic [2:0] alucontrol,
           output logic [31:0] result,
           output logic zero);
    logic [31:0] ib, sum;
    logic sub;
    assign sub = (alucontrol[1:0] == 2'b01);
    assign ib  = sub ? ~b : b;
    assign sum = a + ib + sub;
    always_comb
        case (alucontrol)
            3'b000: result = sum;
            3'b001: result = sum;
            3'b010: result = a & b;
```

```
            3'b011: result = a | b;
            3'b101: result = sum[31];
            default: result = 0;
        endcase
    assign zero = (result == 0);
endmodule
module RegFile(input logic clk, WE3,
               input logic [4:0] A1, A2, A3,
               input logic [31:0] WD3,
               output logic [31:0] RD1, RD2);
    logic [31:0] rf[31:0];
    always_ff @(posedge clk)
        if (WE3 && A3 != 0) rf[A3] <= WD3;
    assign RD1 = (A1 != 0) ? rf[A1] : 0;
    assign RD2 = (A2 != 0) ? rf[A2] : 0;
endmodule

module Extend(input logic [31:0] Instr,
              input logic [1:0] ImmSrc,
              input logic ImmIsU,          // added
              output logic [31:0] ImmExt);
    always_comb begin
        if (ImmIsU) begin
            // U-type (AUIPC): imm[31:12] = Instr[31:12], low 12 bits zero
            ImmExt = { Instr[31:12], 12'b0 };
        end else begin
            case (ImmSrc)
                2'b00: ImmExt = {{20{Instr[31]}}, Instr[31:20]}; // I
                2'b01: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}; // S
                2'b10: ImmExt = {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}; // B
                2'b11: ImmExt = {{12{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:21], 1'b0}; // J
                default: ImmExt = 32'bx;
            endcase
        end
    end
endmodule

module aluDecoder(input logic [2:0] funct3,
                  input logic f7b5,
                  input logic [1:0] aluop,
                  input logic op5,
                  output logic [2:0] alucontrol);
    logic kind;
    assign kind = f7b5 & op5;
```

```
    always_comb
        case (aluop)
            2'b00: alucontrol = 3'b000;
            2'b01: alucontrol = 3'b001;
            default:
                case ({kind, funct3})
                    4'b0000: alucontrol = 3'b000;
                    4'b1000: alucontrol = 3'b001;
                    4'b0111: alucontrol = 3'b010;
                    4'b0110: alucontrol = 3'b011;
                    4'b0010: alucontrol = 3'b101;
                    default: alucontrol = 3'bxxx;
                endcase
        endcase
endmodule

module instructionDecoder(input logic [6:0] op,
                          output logic [1:0] immSrc,
                          output logic ImmIsU);      // added
    always_comb begin
        ImmIsU = 1'b0; // default
        case (op)
            7'b0100011: immSrc = 2'b01;
            7'b1100011: immSrc = 2'b10;
            7'b1101111: immSrc = 2'b11;
            7'b0010111: begin immSrc = 2'b00; ImmIsU = 1'b1; end // AUIPC -> ImmIsU=1, immSrc
set to default (not used when ImmIsU)
            default: immSrc = 2'b00;
        endcase
    end
endmodule

module mainFSM(
    input logic [6:0] op,
    input logic clk, reset, zero,
    output logic pcWrite, regWrite, memWrite,
    output logic irWrite, adrSrc,
    output logic [1:0] resultSource,
    output logic [1:0] aluSourceA,
    output logic [1:0] aluSourceB,
    output logic [1:0] aluOp);

    // added S11 for AUIPC
    typedef enum logic[3:0] {S0,S1,S2,S3,S4,S5,S6,S7,S8,S9,S10,S11} state_t;
```

```
    state_t state, next;
    logic branch, pcUpd;
    assign pcWrite = (branch & zero) | pcUpd;
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else state <= next;
    always_comb begin
        regWrite = 0; memWrite = 0; irWrite = 0; adrSrc = 0;
        resultSource = 0; aluSourceA = 0; aluSourceB = 0;
        aluOp = 0; pcUpd = 0; branch = 0; next = S0;
        case (state)
            S0: begin irWrite=1; aluSourceB=2'b10; resultSource=2'b10; pcUpd=1; next=S1; end
            S1: begin aluSourceA=2'b01; aluSourceB=2'b01;
                case(op)
                    7'b0000011,7'b0100011: next=S2;
                    7'b0110011: next=S6;
                    7'b0010011: next=S8;
                    7'b1101111: next=S9;
                    7'b1100011: next=S10;
                    7'b0010111: next=S11; // added for transition
                endcase
            end
            S2: begin aluSourceA=2'b10; aluSourceB=2'b01; next=(op==7'b0000011)?S3:S5; end
            S3: begin adrSrc=1; next=S4; end
            S4: begin regWrite=1; resultSource=2'b01; next=S0; end
            S5: begin adrSrc=1; memWrite=1; next=S0; end
            S6: begin aluSourceA=2'b10; aluOp=2'b10; next=S7; end
            S7: begin regWrite=1; next=S0; end
            S8: begin aluSourceA=2'b10; aluSourceB=2'b01; aluOp=2'b10; next=S7; end
            S9: begin aluSourceA=2'b01; aluSourceB=2'b10; pcUpd=1; next=S7; end
            S10: begin aluSourceA=2'b10; aluOp=2'b01; branch=1; next=S0; end

            // AUIPC execute + writeback state
            S11: begin
                aluSourceA = 2'b01; // PC
                aluSourceB = 2'b01; // Imm
                aluOp      = 2'b00; // ADD
                regWrite   = 1;
                resultSource = 2'b00; // ALU -> rd
                next       = S0;
            end
        endcase
    end
endmodule
```

```
module controller(
    input logic clk, reset,
    input logic [6:0] op,
    input logic [2:0] funct3,
    input logic funct7b5,
    input logic zero,
    output logic [1:0] immsrc,
    output logic [1:0] alusrcA,
    output logic [1:0] alusrcB,
    output logic [1:0] resultsrc,
    output logic adrsrc,
    output logic [2:0] alucontrol,
    output logic irwrite, pcwrite,
    output logic regwrite, memwrite,
    output logic ImmIsU      // added
    );
    logic [1:0] aluOp;
    // call instruction decoder (now provides ImmIsU)
    instructionDecoder ID(op, immsrc, ImmIsU);
    // main finite-state machine
    mainFSM FSM(op, clk, reset, zero,
                pcwrite, regwrite, memwrite,
                irwrite, adrsrc,
                resultsrc, alusrcA, alusrcB, aluOp);
    // ALU decoder (uses opcode-bit op[5] as before)
    aluDecoder ALD(funct3, funct7b5, aluOp, op[5], alucontrol);
endmodule

module datapath(
    input logic clk, reset,
    input logic [1:0] immsrc,
    input logic ImmIsU,                // added
    input logic [1:0] alusrcA,
    input logic [1:0] alusrcB,
    input logic [1:0] resultsrc,
    input logic adrsrc,
    input logic [2:0] alucontrol,
    input logic irwrite, pcwrite,
    input logic regwrite,
    input logic [31:0] readdata,
    output logic [6:0] op,
    output logic [2:0] funct3,
    output logic funct7b5,
```

```
    output logic zero,
    output logic [31:0] adr, writedata);
    logic [31:0] pc, oldpc, instr, data;
    logic [31:0] ra, rb, a, imm, sa, sb;
    logic [31:0] aluR, aluOut, res;
    flopr2 #(32) PC(clk, reset, pcwrite, res, pc);
    mux2  #(32) AM(pc, res, adrsrc, adr);
    flopr2 #(32) OPC(clk, reset, irwrite, pc, oldpc);
    flopr2 #(32) IR(clk, reset, irwrite, readdata, instr);
    flopr  #(32) DR(clk, reset, readdata, data);
    assign op      = instr[6:0];
    assign funct3  = instr[14:12];
    assign funct7b5 = instr[30];
    RegFile RF(clk, regwrite, instr[19:15], instr[24:20], instr[11:7], res, ra, rb);
    // NOTE: Extend now takes full instr and ImmIsU
    Extend EX(instr, immsrc, ImmIsU, imm);
    flopr #(32) AR(clk, reset, ra, a);
    flopr #(32) BR(clk, reset, rb, writedata);
    mux3 #(32) SA(pc, oldpc, a, alusrcA, sa);
    mux3 #(32) SB(writedata, imm, 32'd4, alusrcB, sb);
    alu U(sa, sb, alucontrol, aluR, zero);
    flopr #(32) AO(clk, reset, aluR, aluOut);
    mux3 #(32) RM(aluOut, data, aluR, resultsrc, res);
endmodule

module memory(
    input logic clk, WE,
    input logic [31:0] A, WD,
    output logic [31:0] RD);
    logic [31:0] RAM[63:0];
    initial $readmemh("memfile.dat", RAM);
    assign RD = RAM[A[31:2]];
    always_ff @(posedge clk)
        if (WE) RAM[A[31:2]] <= WD;
endmodule

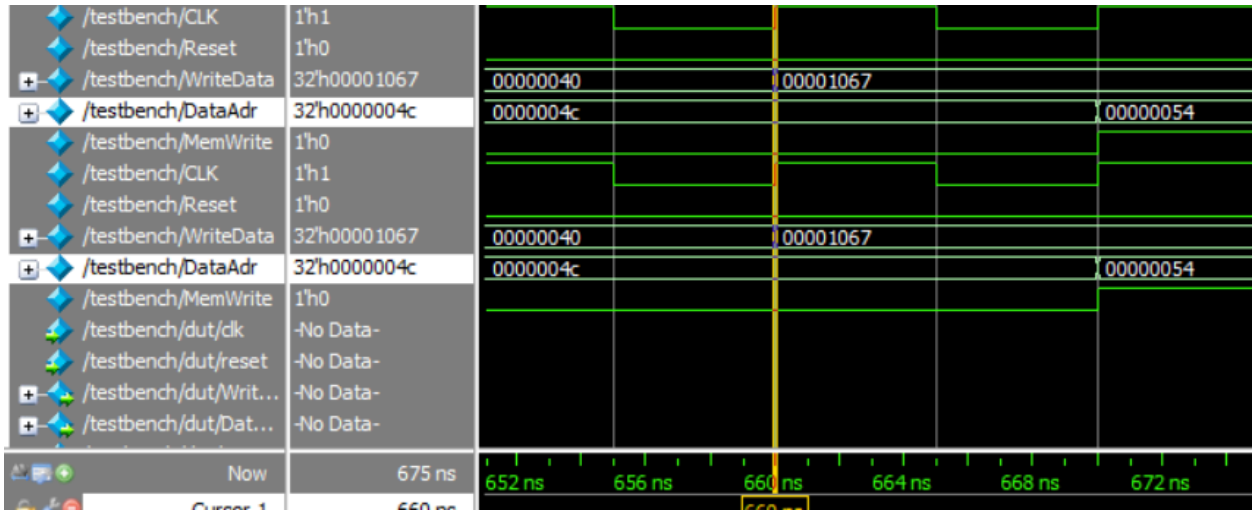
module risc_Processor(
    input logic clk, reset,
    input logic [31:0] readdata,
    output logic memwrite,
    output logic [31:0] adr, writedata);
    logic pcw, as, is, rw;
    logic [1:0] rs, aA, aB, ims;
    logic [2:0] ac;
```

```
    logic [6:0] op;
    logic [2:0] f3;
    logic f7, z;
    logic ImmIsU; // added

    datapath DP(clk, reset, ims, ImmIsU, aA, aB, rs,
                as, ac, is, pcw, rw,
                readdata, op, f3, f7, z,
                adr, writedata);
    controller CT(clk, reset, op, f3, f7, z,
                  ims, aA, aB, rs,
                  as, ac, is, pcw, rw, memwrite, ImmIsU);
endmodule

module top(
    input logic clk, reset,
    output logic [31:0] WriteData, DataAdr,
    output logic MemWrite);
    logic [31:0] ReadData;
    risc_Processor CPU(clk, reset, ReadData, MemWrite, DataAdr, WriteData);
    memory DM(clk, MemWrite, DataAdr, WriteData, ReadData);
endmodule
```

Wave form showing 0x1067 being written into location 84



All waveforms

